Computer Organization and Assembly Language

Week 11 — Chapter 5: Subroutines

Dr. Maqsood M. Khan

Department of Computer Science / Software Engineering / Artificial Intelligence
FAST-NUCES Peshawar

Week 11

Outline

1. Quick Review
2. The Problem: Code Reuse
3. CALL and RET
4. Example 5.1 — First Subroutine
5. Example 5.2 — Calling Twice
6. The Shortcoming

Quick Review: What Was Done Last Week?

- **Logical operations are bitwise:** AND, OR, XOR, NOT process each bit independently and in parallel.
- **AND** clears bits — mask 0 $\rightarrow$ force to 0, mask 1 $\rightarrow$ keep original.
- **OR** sets bits — mask 1 $\rightarrow$ force to 1, mask 0 $\rightarrow$ keep original.
- **XOR** flips bits — reversible: $(A \oplus B) \oplus B = A$. Foundation of cryptography.
- **NOT** inverts all bits, no mask, no flags changed.
- **TEST** = non-destructive AND — sets flags only, neither operand changes. Use `JZ`/`JNZ` after.
- **Example 4.3:** mask in BX slides left each loop to test each multiplier bit without destroying `[multiplier]`.
- AND/OR/XOR always clear CF and OF — never use `JC` after these.

Why Do We Need Subroutines?

The situation so far

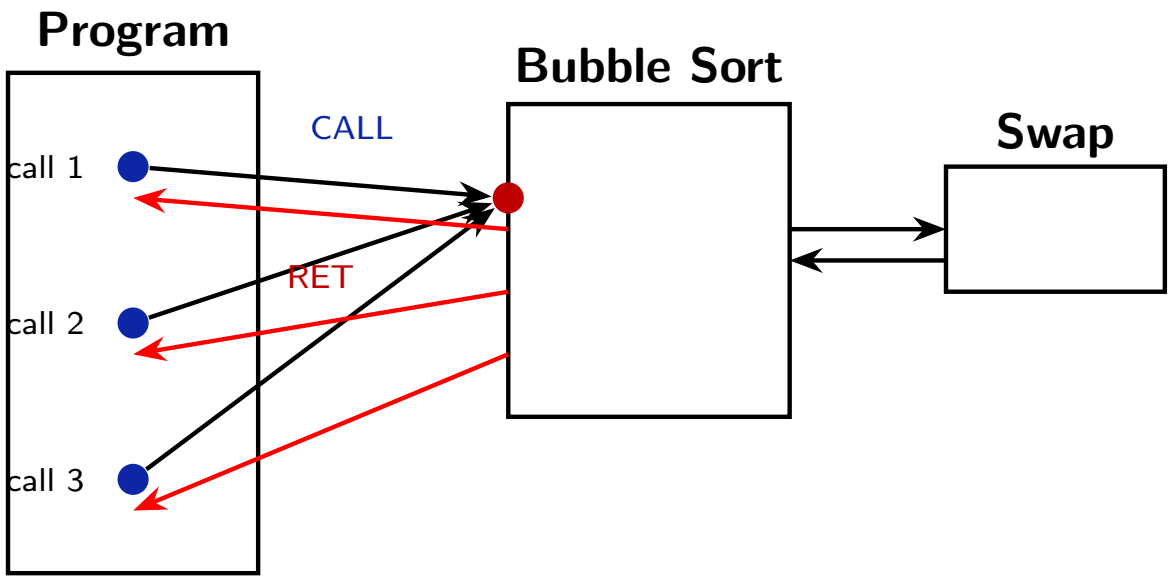
We have written two powerful algorithms: **bubble sort** and **multiplication**. A real program may need to sort or multiply at **100 different places**.

- **Copy the code 100 times?** Totally infeasible — code size explodes and maintenance becomes impossible.
- **Write at one place with a label and JMP to it?** Better — but after sorting is done, *how does the processor know where to go back?*
- **JMP back to a label after the call?** JMP has a single fixed target. We jumped from 100 places — which one do we return to?

The core problem

JMP is a permanent diversion. Like a turn on a highway with no return. We need a **temporary diversion** — go somewhere, do the work, come back to exactly where we left from.

Permanent vs Temporary Diversion



- **Black arrows** = CALL — jump to the subroutine
- **Red arrows** = RET — return to exactly where we left
- The subroutine is written **once** and called from **any number of places**

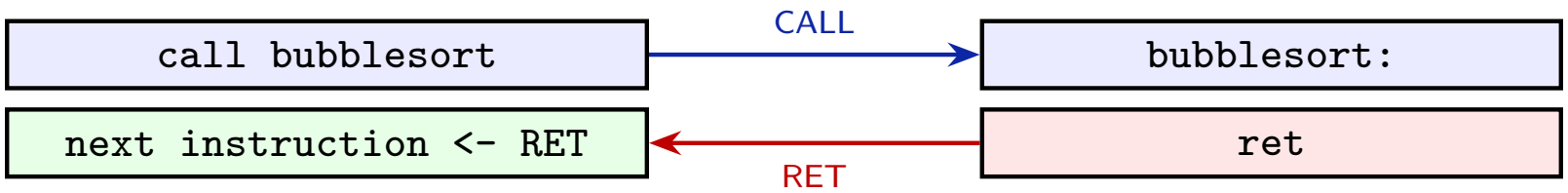
CALL and RET — Temporary Diversion

CALL instruction

Takes a **label** as argument (just like JMP). Diverts execution to that label but **saves the return address** on the stack so it can come back.

RET instruction

Takes **no arguments**. Pops the return address from the stack into IP — execution resumes at the instruction **following** the CALL.



Key point — CALL and RET are a logical pair

CALL places the return address on the stack; RET picks it up and jumps back. Each CALL from a different place returns to its own correct location.

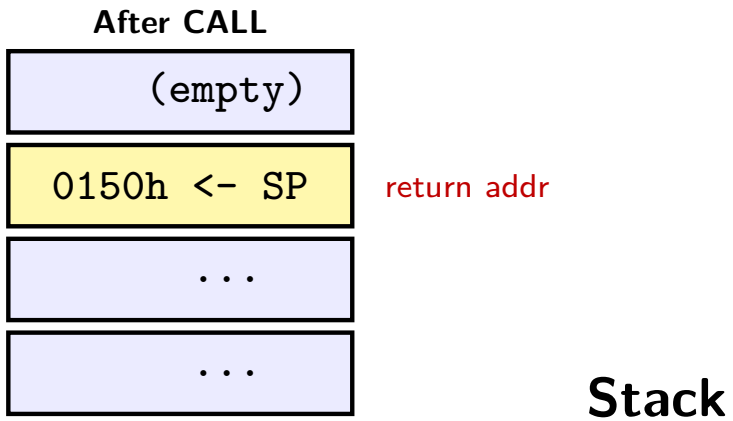
How the Stack Coordinates CALL and RET

- When `call bubblesort` executes:
  1. SP is **decremented by 2**
  2. The address of the **next instruction** (return address) is **pushed onto the stack**
  3. IP is set to `bubblesort` label
- When `ret` executes:
  1. The return address is **popped from the stack** into IP
  2. SP is **incremented by 2**
  3. Execution resumes from the saved address

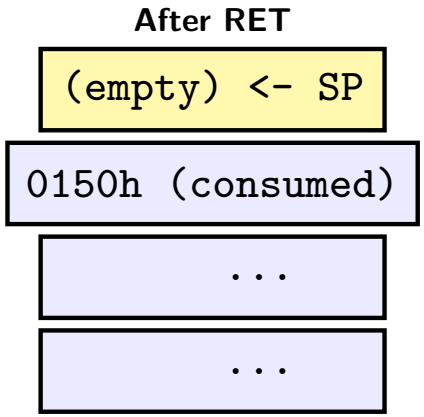
Why this works for 100 calls

Each CALL pushes a *different* return address. Each RET pops the correct one — every call returns to its own caller.

Stack during CALL



after RET



Parameters — Passing Data to a Subroutine

The problem

If we always call `bubblesort`, it will always sort the *same* array in the *same* order. In a real application we need to sort **different arrays of different sizes**.

- Information that changes from call to call is called a **parameter**.
- Parameters must be **passed from the caller** to the subroutine before the CALL.
- CALL does not change registers (except IP and SP), so **registers are the natural place to pass parameters**.

Register	Parameter passed
`BX`	Start address of the array
`CX`	Number of elements in the array

Key point

The caller loads parameters into registers **before** the CALL. The subroutine reads them from those registers.

Example 5.1 — Bubble Sort as a Subroutine

```
1  ; bubble sort algorithm as a subroutine
2  [org 0x0100]
3          jmp start
4
5  data: dw 60, 55, 45, 50, 40, 35, 25, 30, 10, 0
6  swap: db 0
7
8  bubblesort:
9          dec cx ; last element not compared
10         shl cx, 1 ; turn into byte count
11 mainloop: mov si, 0 ; initialize array index to zero
12         mov byte [swap], 0 ; reset swap flag to no swaps
13 innerloop:
14         mov ax, [bx+si] ; load number in ax
15         cmp ax, [bx+si+2] ; compare with next number
16         jbe noswap ; no swap if already in order
17         mov dx, [bx+si+2] ; load second element in dx
18         mov [bx+si], dx ; store first number in second
19         mov [bx+si+2], ax ; store second number in first
20         mov byte [swap], 1 ; flag that a swap has been done
21 noswap: add si, 2 ; advance si to next index
22         cmp si, cx ; are we at last index?
23         jne innerloop ; if not, compare next two
24         cmp byte [swap], 1 ; check if a swap has been done
25         je mainloop ; if yes, make another pass
26         ret ; go back to where we came from
27
28 start: mov bx, data ; send start of array in bx
29         mov cx, 10 ; send count of elements in cx
30         call bubblesort ; call our subroutine
31         mov ax, 0x4c00
32         int 0x21
```

Example 5.1 — Explained

The subroutine

Label `bubblesort:` the anchor point. Everything above `start` is the subroutine code.

`dec cx:` one less comparison than elements.
`shl cx, 1:` convert element count to *byte* count (word array, 2 bytes each).

`ret:` at the end — returns to caller.

The main program

`mov bx, data:` load array address into BX (the parameter).
`mov cx, 10:` load count into CX (the parameter).
`call bubblesort:` invoke the subroutine.

After the CALL returns, execution continues at `mov ax, 0x4c00`.

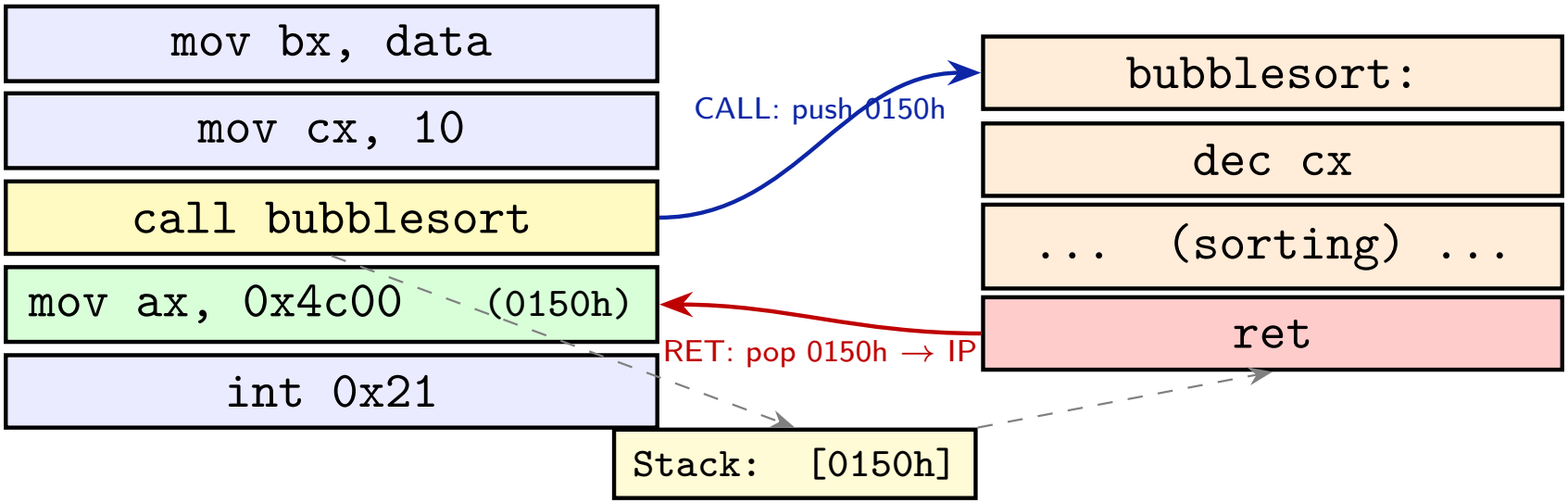
Parameters received

BX = address `0103h` (start of data)
CX = `000Ah` = 10 (element count)

Debugger observation

Step **over** CALL → data sorted in one step.
Step **into** CALL → lands at `bubblesort`.
At RET → SP recovers, `0150h` → IP.

What Happens at CALL and RET — Step by Step



- CALL pushes `0150h` (address of next instruction) onto stack and jumps to `bubblesort`
- RET pops `0150h` from stack into IP — execution resumes at `0150h`

Example 5.2 — Same Subroutine, Two Arrays

Data (new):

```
data: dw 60,55,45,50,40
      dw 35,25,30,10,0
data2: dw 328,329,898,8923
      dw 8293,2345,10,877
      dw 355,98,888,533
      dw 2000,1020,30,200
      dw 761,167,90,5
swap: db 0
```

Subroutine — identical to 5.1:

```
bubblesort:
  dec cx / shl cx,1
  ... (same sorting code) ...
  ret
```

Main program (two calls):

```
start:
  mov bx, data ; array 1
  mov cx, 10 ; 10 elems
  call bubblesort

  mov bx, data2 ; array 2
  mov cx, 20 ; 20 elems
  call bubblesort

  mov ax, 0x4c00
  int 0x21
```

Key change

Same subroutine, two calls.
Different BX and CX passed each time.
Subroutine code unchanged.

Example 5.2 — What Changed and Why

Two data arrays

`data:` 10 elements (word array).
`data2:` 20 elements declared on two lines — they are *consecutive in memory*, no extra label needed.

The subroutine code is **identical** to Example 5.1 — not a single line changed.

Two calls in main

First call: BX = `data`, CX = 10
Second call: BX = `data2`, CX = 20

Different parameters → different array sorted each time. Same subroutine used both times.

Debugger — two different return addresses

First CALL: pushes `0178h` onto stack.
RET pops `0178h` → IP.

Second CALL: pushes `0181h` onto stack.
RET pops `0181h` → IP.

Each call gets its own return address — this is how the stack allows the same subroutine to be called from anywhere.

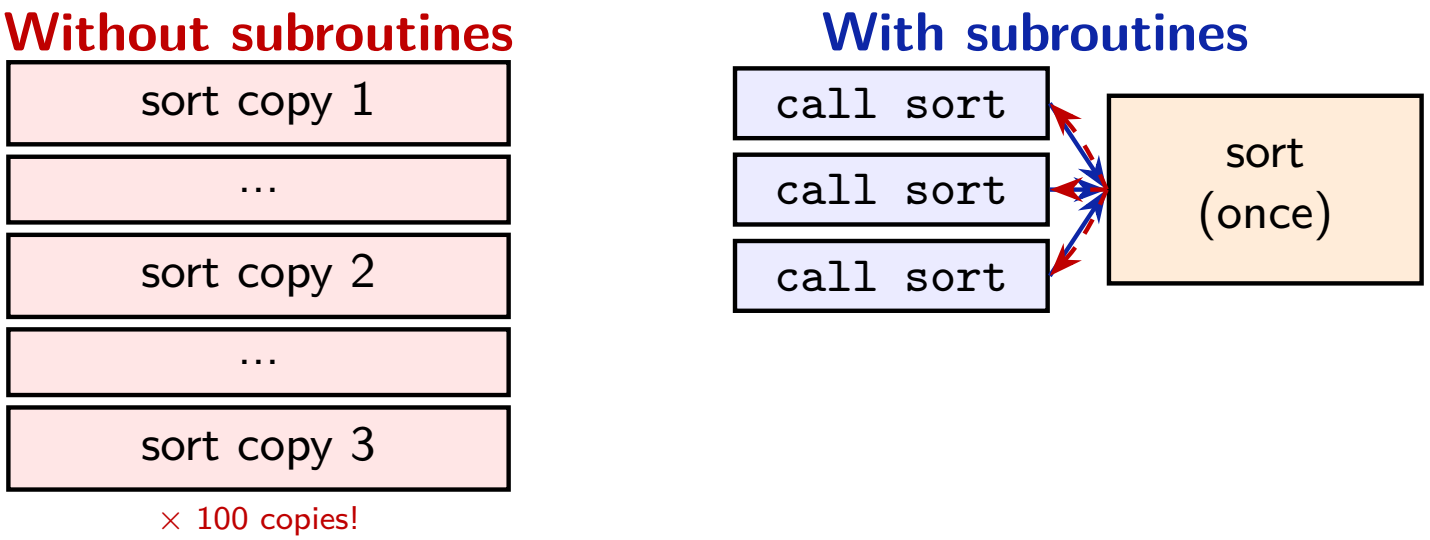
Stack after 1st CALL

```
0178h ← 1st call
```

Stack after 2nd CALL

```
0181h ← 2nd call
```

The Reusability Gain — Visualised



- **Left (bad):** same sorting code copied 100 times — 100× the size, impossible to maintain.
- **Right (good):** sort written once, called 100 times — each call returns to its own caller automatically.

The Shortcoming — Register Destruction

What the subroutine destroys

The bubble sort subroutine uses and modifies: **AX, CX, DX, SI** — all overwritten during sorting.

- After `call bubblesort` returns, these registers contain **meaningless data** from the sorting process.
- The caller must **not hold important data** in AX, CX, DX, or SI before calling — otherwise it is lost forever.
- With **thousands of subroutines** in a real program, expecting the caller to memorise which registers each subroutine destroys is **unrealistic and unreasonable**.
- Registers are **limited in number** — restricting the caller on register use makes the caller's job very difficult.

The solution

This shortcoming will be removed using the **system stack** — the subroutine saves registers at entry and restores them before returning. The caller sees no change. This will be covered in the next topic.

Summary

- **Problem:** code needed at 100 places cannot be copied 100 times — size and maintenance make it impossible.
- **JMP alone is not enough:** it is a permanent diversion with no concept of returning.
- **CALL** = temporary diversion. Saves the return address on the stack, then jumps to the label.
- **RET** = return. Pops the return address from the stack into IP — execution resumes after the CALL.
- **Parameters** are passed in registers (BX, CX) before the CALL. CALL does not change registers except IP and SP.
- **Example 5.1:** bubblesort written once, called once with BX = array address, CX = element count.
- **Example 5.2:** same subroutine called twice with different arrays — each CALL pushes its own return address; each RET returns to the correct caller.
- **Shortcoming:** subroutine destroys AX, CX, DX, SI. Fix: use the **system stack** to save and restore registers — coming next.

Thank You

Any Questions?

"Assembly language only gives you the basic building blocks. Whether you build a hut or a palace from them depends only on your imagination."

Next: The System Stack — `PUSH`, `POP` and saving registers in subroutines